

A Typesetting System to Untangle the Scientific Writing Process

Filippo Marcon

University of Trento

filippo.marcon@studenti.unitn.it

Abstract—The process of scientific writing is often tangled up with the intricacies of typesetting, leading to frustration and wasted time for researchers. In this paper, we introduce Typst, a new typesetting system designed specifically for scientific writing. Typst untangles the typesetting process, allowing researchers to compose papers faster. In a series of experiments we demonstrate that Typst offers several advantages, including faster document creation, simplified syntax, and increased ease-of-use.

Index Terms—Scientific writing, Typesetting, Document creation, Syntax

I. INTRODUCTION

Sparse Matrix Vector multiplication (SpMV) is a fundamental problem in many computing fields. It is highly parallelizable and its performance is bounded by memory bandwidth. Formally, SpMV is defined as the multiplication between A , a sparse matrix of size $m \times n$ and x , a dense vector of size n . The resulting vector y is of size m .

$$y = Ax \quad (1)$$

This investigation focuses on relatively basic, non-adaptive GPU SpMV kernels, both for CSR and COO. We address these questions:

- What is the penalty of the additional storage requirements of COO?
- Is this penalty balanced by the ability to split work based on non-zeros without additional preprocessing?
- How much does memory coalescing improve the performance of the kernels?
- What are the ideal launch parameters for each of the kernels? (can we use cuda tool to have them auto)
- How does matrix size, non-zeros per row, structure affect the performance of the kernels?
- How do these kernels compare to the CuSparse library performance?
- What is the memory/cache usage of our kernels?

II. METHODOLOGY

The sparse formats used in the study are CSR and COO. COO is an obvious starting point as it's the simplest storage format for sparse matrices, and it's used in the SuiteSparse dataset (see Section III.A). COO uses three arrays to store the matrix: for the n -th non-zero element of the matrix `rows[n]` indicates the row where it is located, `cols[n]` indicates the column, and `values[n]` stores the actual value. CSR still uses three arrays, with the only change being that the rows array is compressed with a prefix sum. The compression is

possible only if the non-zero elements are sorted by row index. The CSR format takes less storage than COO, however CSR-based SpMV algorithms that split tasks by row can suffer from load imbalance on sparse matrices with irregular nnz distribution along rows[1].

The CPU implementation is a simple iteration over all the non-zero elements. Its only goal is to provide a reference result to measure the error of the GPU implementations.

```
for each row i in [0, m):  
    for each nonzero j in [rows[i], rows[i+1]):  
        y[i] ← y[i] + vals[j] * x[cols[j]]
```

Listing 1. Naive CPU implementation

Moving to the GPU kernels, we assume that `tid = blockIdx.x * blockDim.x + threadIdx.x`. One of the simplest ways to parallelize SpMV with COO is to use one thread per non-zero. This algorithm, commonly called *COO_flat*, assigns one thread for each non-zero element, then uses atomic adds to write the result to y . This approach offers maximum parallelism and load balancing, and a certain degree of coalescing as the COO is stored in row-major order. On the other hand the `atomicAdd` hurts the performance a lot, and with a high enough nnz we are not able to launch all threads together.

```
if tid < nnz:  
    atomicAdd(y[rows[tid]], vals[tid] *  
             x[cols[tid]])
```

Listing 2. COO flat kernel

As an improvement to the shortcomings of the flat COO kernel we propose an adaptation of the COO algorithm described by Bell et.al.[2]. Computation is divided in *chunks*, which can span multiple rows, each chunk has a warp(32 threads) assigned to it. The threads stride over the length of the chunk together, to guarantee memory coalescing. On each step the threads first calculate the multiplication for their assigned non-zero and then they perform a *segmented reduction* to aggregate the results. The aggregated value is carried along through the `carry` register until a new row is found, then one thread writes the aggregated result to memory. This method drastically reduces the conflicts caused by `atomicAdd`, while guaranteeing memory coalescing and balanced distribution of non-zeros. The biggest difference from the algorithm presented by Bell et.al. is the usage of the more modern `shfl` instructions for the segmented reduction instead of using shared memory.

```

carry_row ← -1
carry_val ← 0

for each tile of 32 nonzeros starting at n in
[chunk_start, chunk_end]:

    Each lane i loads nonzero (row[n+i], val[n+i] *
x[col[n+i]])
    Inactive lanes (beyond chunk_end) load row = -1,
val = 0

    // --- Merge with carry from previous tile
    Lane 0 checks its row against carry_row:
    if row == carry_row:
        val ← val + carry_val
    else:
        atomicAdd(y[carry_row], carry_val)

    // --- segmented prefix sum (via shuffle)
    for offset = 1, 2, 4, 8, 16:
        left_row, left_val ← values from lane (i -
offset)
        if left_row == row:
            val ← val + left_val

    // --- Write completed rows, save carry
    next_row ← row of lane (i + 1)
    last_lane ← last active lane in tile

    if lane == last_lane:
        carry_row, carry_val ← row, val
    else if row ≠ next_row:
        atomicAdd(y[row], val)

    carry_row, carry_val ← values held by last_lane
(warp broadcast)

// --- Flush final carry
Lane 0: atomicAdd(y[carry_row], carry_val)

```

Listing 3. COO segmented reduction kernel

Moving to CSR format, we start with the *CSR scalar* kernel, which assigns one thread per matrix row. The thread then sequentially computes the multiplication between the entire row and the corresponding value of x and stores it. This basic approach suffers from load imbalance proportionally to the distribution of non-zeros per row of the matrix. It also has no memory coalescing, and using only one thread per row is an underutilization of GPU potential, unless the matrix is very large and with a very small amount of non-zeros per row.

```

if tid < num_rows:
    sum ← 0
    for each nonzero j in [rows[tid], rows[tid+1]):
        sum ← sum + vals[j] * x[cols[j]]
    y[tid] ← sum

```

Listing 4. CSR scalar kernel

To solve most of the issues of the scalar kernel we implement the *CSR_vector* kernel, which assigns a warp per matrix row. Now similarly to *COO_segmented* we have memory coalescing thanks to the threads striding together along the row. We also have a similar reduction, which is made simpler by the fact that we are not spanning rows, so we can simply use `shfl` to aggregate the 32 results and then make thread 0 write them to memory. The problem that remains unsolved is that we are still splitting by row, which makes the performance vulnerable to matrices with high variance of non-zeros per row.

```

if row < num_rows:
    sum ← 0
    for each nonzero j in [rows[row] + lane,
rows[row+1]) with stride 32:
        sum ← sum + vals[j] * x[cols[j]]

    for offset = 16, 8, 4, 2, 1:
        sum ← sum + value from lane (i + offset)

    if lane == 0:
        y[row] ← sum

```

Listing 5. CSR vector kernel

A. Validation method

First the result y_c is calculated using the algorithm in Listing 1, then for each execution of a kernel the result y_k is compared by dividing the l2 norm of the difference by the l2 norm of the reference. For all methods the error does not reach magnitudes over 10^{-7} .

$$\text{err} = \frac{\|y_c - y_k\|_2}{\|y_c\|_2} \quad (2)$$

B. Measurement methodology

Each combination of kernel and launch parameters (block and grid size) is benchmarked by first launching the kernel WARMUP times without logging the results, and then by running it another NITER times, then computing the average of each measurement across runs. The parameters WARMUP, NITER are user-defined, and were set respectively to 2 and 10 during all the experiments.

Timings are measured through Cuda Events, and only the kernel execution time is measured. FLOP/s are measured by dividing the number of required floating point operations ($\text{nnz} * 2$) by the arithmetic mean of the execution time across the NITER runs.

C. Hardware/Software environment

The experiments were run on the unitn *Baldo* cluster on a *NVIDIA L40s 48GB* graphic card. The program was compiled with *gcc* version 13.3.0 and *cuda* version 12.5.0 .

III. DATASET

A. Sparse Matrices

The dataset chosen for benchmarking the various kernels is a subset of the 14 matrices selected by S. Williams et al. [3] and later also used in a NVIDIA technical report [4], hosted on the SuiteSparse Matrix Collection [5]. This small selection consists of matrices derived from real-world problems in different fields. The matrices are intentionally varied in dimension, non-zeros per row, existence of dense block structure, and degree of non-zero concentration along the diagonal. Table I provides a summary of the dataset.

TABLE I
SUMMARY OF MATRIX SELECTION

Matrix	Rows	Cols	NNZ
Protein	36,417	36,417	4,344,765
FEM/Spheres	83,334	83,334	6,010,480
FEM/Cantilever	62,451	62,451	4,007,383
FEM/Harbor	46,835	46,835	2,374,001
FEM/Ship	140,874	140,874	7,813,404
Economics	206,500	206,500	1,273,389
Epidemiology	525,825	525,825	2,100,225
FEM/Accelerator	121,192	121,192	2,624,331
Circuit	170,998	170,998	958,936
Webbase	1,000,005	1,000,005	3,105,536

B. Parsing

The .mtx format used by SuiteSparse stores matrices in COO, with column-major order. The parsing is done through the library fast-matrix-market, as the .mtx format allows a number of different representations to save space(standard, symmetric, skew-symmetric) meaning that writing a parser from scratch could lead to errors and/or distract from the main goal. After parsing we sort the matrix in row-major order, and then create a new vector with the rows in CSR format. The result is that we use row-major ordering for both COO and CSR, for consistence and ease of use when writing and comparing kernels.

C. Input Vector

The Float32 Input Vector is randomly generated with a fixed seed to guarantee reproducibility across runs. The user-defined parameter MAX_VECTOR_VALUE defines the upper bound to the randomly generated values.

IV. RESULTS

Fig. 1 shows the average performance of each kernel across all 10 matrices, grouped by the experimented block size. The red dotted line shows the average performance of the CuSparse library over all of the matrices. We can see that starting from 128 threads per block the performance of all kernels stabilizes, and increasing past that number does not improve, and in some cases worsens the performance.

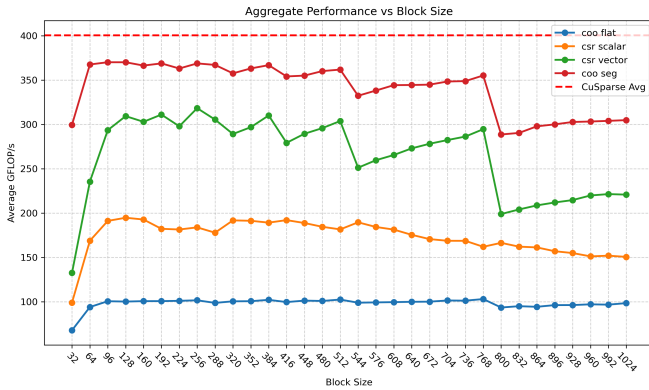


Fig. 1. Aggregate Performance related to Block Size

Fig. 2, Fig. 3, Fig. 4 shows the best performance with a fixed block size of 128 of the 4 kernels for each matrix. In the

three plots the matrices are respectively sorted by increasing density, variance of nnz per row, and total nnz. We use line plots instead of bar plots to better visualize trends across the 10 matrices.

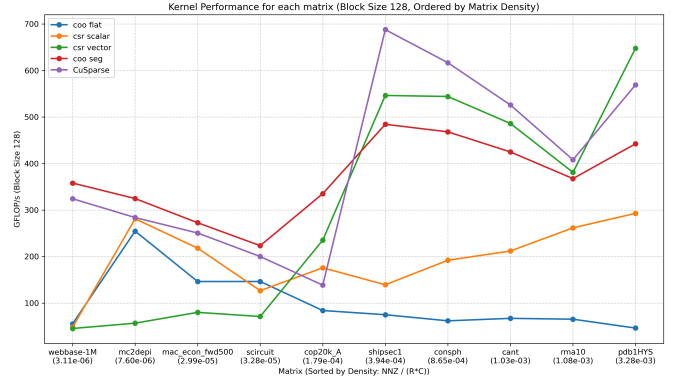


Fig. 2. Kernel best performance for each matrix, sorted by matrix density

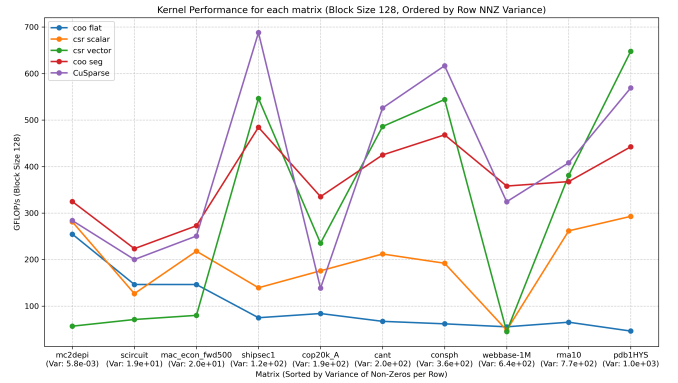


Fig. 3. Kernel best performance for each matrix, sorted by variance of nnz per row

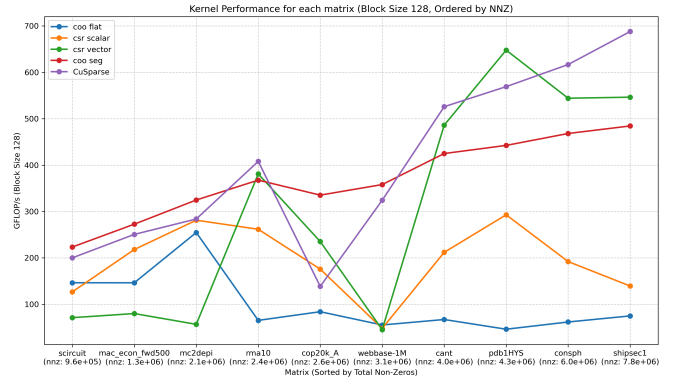


Fig. 4. Kernel best performance for each matrix, sorted by total nnz

We briefly attempted to measure cache usage by choosing a fixed block size, and profiling one kernel at a time with `ncu` but found out that the unitn *Baldo* cluster did not allow access to NVIDIA GPU performance counters. Not having access to other GPUs we had to abandon the profiling attempt.

V. DISCUSSION

VI. CONCLUSION

COO flat, is the worst performing algorithm on average. It performs best with the very small matrices, dropping down quickly on larger.

REFERENCES

- [1] J. Gao, B. Liu, W. Ji, and H. Huang, "A Systematic Literature Survey of Sparse Matrix-Vector Multiplication." [Online]. Available: <https://arxiv.org/abs/2404.06047>
- [2] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of the conference on high performance computing networking, storage and analysis*, 2009, pp. 1–11.
- [3] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, "Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms," *Parallel Computing*, vol. 35, no. 3, pp. 178–194, Mar. 2009, doi: 10.1016/j.parco.2008.12.006.
- [4] N. Bell and M. Garland, "Efficient Sparse Matrix-Vector Multiplication on CUDA," NVIDIA Technical Report NVR-2008–4, Dec. 2008.
- [5] T. A. Davis and Y. Hu, "The university of Florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, Dec. 2011, doi: 10.1145/2049662.2049663.